1.)

```java
public class ComprobarPrimo {
    private int numero;

    public ComprobarPrimo(int numero) {
        this.numero = numero;
    }

    public boolean ePrimo() {
        for (int n = 2; n * n < numero; n++) {
            if (numero % n == 0) {
                return false;
            }
        }
        return true;
    }
}
```

1(Default)+1(for)+1(if)

2)

```java
public class Factorial {
    public float factorial(byte n) throws Exception {
        if (n < 0) {
            throw new Exception(                                    );
        }
        float resultado = 1;
        for (int i = 2; i <= n; i++) {
            resultado *= i;
        }
        return resultado;
    }
}
```

1(Default)+1(if)+1(for)

3)

Una empresa de gestión de cursos tiene un sistema que clasifica la nota final de un alumno según su puntuación (de 0 a 10):

- “Menor que 5 → “Suspenso””
- “Entre 5 y 6,9 → “Aprobado””
- “Entre 7 y 8,9 → “Notable””
- “Entre 9 y 10 → “Sobresaliente””

Diseña pruebas de **caja negra** para comprobar que el sistema funciona correctamente.

En tu respuesta:

1. “Identifica las **clases de equivalencia**”
2. “Indica los **valores límite** más importantes”
3. “Propón al menos **5 casos de prueba concretos** (entrada y salida esperada)”

1. [0, 5)[5, 6.9][7, 8.9][9, 10]

2.
(4.9 , 5) – Fail-to-Pass limit
(6.9 , 7) – Pass-to-Notable limite
(8.9 , 9) – Notable-to-Sobresaliente limite
(0 , 10) – Edgecases

3.
2,5 – To test Fail
5 – Pass edge
6.9 – Pass-Before-Notable edge
7.5 – Notable case
10 – Sobresaliente-top-edge
-1 , 11 – Error edges

Dado el siguiente método:

‹/› Java

```java
public String clasificarNumero(int n) {
    if (n > 0) {
        if (n % 2 == 0) {
            return "Positivo par";
        } else {
            return "Positivo impar";
        }
    } else if (n < 0) {
        return "Negativo";
    } else {
        return "Cero";
    }
}
```

Responde a las siguientes cuestiones:

1. "Dibuja o describe el **flujo de ejecución** del método"
2. "Indica cuántos **caminos independientes** tiene el programa"
3. "Diseña los **casos de prueba necesarios** para cubrir todos los caminos"

1)
1. Checks if n is higher than 0 (positive){
    1.1 Checks if n is pair
}
2. Checks if n is lower than 0 (negative)

2)
Tiene 3 caminos independientes (1st IF, IF inside the 1st IF, ELSE IF)

3)
n = 0
n = 2
n = 3
n = -1